

Project: Derivative Pricing Engine

Jun Shen

July 20, 2026

Contents

1	Why I Wrote This Document	3
1.1	My Notation	3
2	Black-Scholes-Merton Model	3
2.1	The d_1 and d_2 Terms	3
2.2	Price	3
2.3	Greeks	4
2.4	Put-Call Parity	4
2.5	No-Arbitrage Bounds	4
3	Black-76 Model	4
3.1	The d_1 and d_2 Terms	4
3.2	Price	4
3.3	Greeks	5
3.4	Rho: A Shortcut I Rely On	5
4	Binomial Trees: CRR and Leisen-Reimer	5
4.1	Cox-Ross-Rubinstein (CRR) Parameters	5
4.2	Leisen-Reimer (LR) Parameters	5
4.3	Backward Induction and Early Exercise	6
4.4	Greeks From the Tree	6
4.5	Early Exercise Boundary	6
5	Monte Carlo Engine	7
5.1	Geometric Brownian Motion Discretization	7
5.2	Antithetic Variates	7
5.3	Discounted Price and Standard Error	7
5.4	Payoffs I Implement	7
5.5	Basket Options and Correlated Paths	8
5.6	Greeks by Finite Differences (Bump-and-Reprice)	8
6	Finite Difference PDE for Barrier Options	8
6.1	The Black-Scholes PDE	8
6.2	Explicit Finite Difference Discretization	8
6.3	Stability Condition	9
6.4	Boundary and Barrier Conditions	9
6.5	Knock-In via In/Out Parity	9
6.6	Greeks From the Grid	9

7	Implied Volatility Calibration	10
7.1	Newton-Raphson	10
7.2	Bisection Fallback	10
7.3	Rejecting Impossible Prices	10
8	Scenario Analysis	10
9	Model Validation	10
10	Limitations I Am Aware Of	11
11	Future Enhancements I Would Consider	11

1 Why I Wrote This Document

In this project, I built a Python derivatives analytics platform that prices seven different option types using five different pricing techniques (a closed-form Black-Scholes-Merton formula, a closed-form Black-76 formula, two binomial tree constructions, a Monte Carlo simulation engine, and an explicit finite difference PDE solver). Rather than scattering the math across code comments, I collected every formula I actually implemented into this single document, in the same order I built the code. My goal is that anyone reviewing my project can read this document side by side with the corresponding Python file and verify that what I coded matches what I claim I coded.

1.1 My Notation

Throughout this document I use the following symbols consistently:

- S : spot price of the underlying asset (or F for a futures price)
- K : strike price
- T : time to expiry, in years
- r : continuously compounded risk-free rate
- q : continuous dividend yield
- σ : annualized volatility of the underlying
- $N(\cdot)$: cumulative distribution function of the standard normal distribution
- $n(\cdot)$: probability density function of the standard normal distribution

2 Black-Scholes-Merton Model

I use this model for the "European Call and Put" product bucket. It is implemented in `src/models/black_scholes.py`.

2.1 The d_1 and d_2 Terms

Every formula in this section is built from two auxiliary quantities:

$$d_1 = \frac{\ln(S/K) + (r - q + \frac{1}{2}\sigma^2)T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T} \quad (1)$$

I compute these once per pricing call in my `d1_d2()` helper function in `src/models/math_utils.py`, and every other formula in this section reuses that same pair.

2.2 Price

$$C = Se^{-qT}N(d_1) - Ke^{-rT}N(d_2) \quad (2)$$

$$P = Ke^{-rT}N(-d_2) - Se^{-qT}N(-d_1) \quad (3)$$

2.3 Greeks

$$\Delta_{call} = e^{-qT} N(d_1) \quad (4)$$

$$\Delta_{put} = -e^{-qT} N(-d_1) \quad (5)$$

$$\Gamma = \frac{e^{-qT} n(d_1)}{S\sigma\sqrt{T}} \quad (\text{same for calls and puts}) \quad (6)$$

$$\mathcal{V} = Se^{-qT} n(d_1) \sqrt{T} \quad (\text{same for calls and puts, "Vega"}) \quad (7)$$

$$\Theta_{call} = -\frac{Se^{-qT} n(d_1) \sigma}{2\sqrt{T}} - rKe^{-rT} N(d_2) + qSe^{-qT} N(d_1) \quad (8)$$

$$\Theta_{put} = -\frac{Se^{-qT} n(d_1) \sigma}{2\sqrt{T}} + rKe^{-rT} N(-d_2) - qSe^{-qT} N(-d_1) \quad (9)$$

$$\rho_{call} = KTe^{-rT} N(d_2) \quad (10)$$

$$\rho_{put} = -KTe^{-rT} N(-d_2) \quad (11)$$

I report Θ as a per-year quantity and divide by 365 whenever I want per-calendar-day time decay, and I report $\mathcal{V}$ and ρ per unit change in σ and r respectively (I scale these down by 0.01 or 0.0001 in my reporting layer whenever I want "per vol point" or "per basis point" units).

2.4 Put-Call Parity

I rely on this identity throughout my model validation suite:

$$C - P = Se^{-qT} - Ke^{-rT} \quad (12)$$

2.5 No-Arbitrage Bounds

For a European option, I never expect a fair price outside these bounds:

$$\max(Se^{-qT} - Ke^{-rT}, 0) \leq C \leq Se^{-qT} \quad (13)$$

$$\max(Ke^{-rT} - Se^{-qT}, 0) \leq P \leq Ke^{-rT} \quad (14)$$

I use these bounds in two places: to reject an obviously bad market price before running my implied volatility solver, and as a sanity check in my model validation test suite.

3 Black-76 Model

I use this model for the "Options on Futures" product bucket, implemented in `src/models/black76.py`. I treat the futures price F the same way I treated S in Black-Scholes-Merton, except that I set the cost of carry to zero, since a futures price already embeds any carry.

3.1 The d_1 and d_2 Terms

$$d_1 = \frac{\ln(F/K) + \frac{1}{2}\sigma^2 T}{\sigma\sqrt{T}}, \quad d_2 = d_1 - \sigma\sqrt{T} \quad (15)$$

3.2 Price

$$C = e^{-rT} [FN(d_1) - KN(d_2)] \quad (16)$$

$$P = e^{-rT} [KN(-d_2) - FN(-d_1)] \quad (17)$$

3.3 Greeks

$$\Delta_{call} = e^{-rT} N(d_1) \quad (18)$$

$$\Delta_{put} = -e^{-rT} N(-d_1) \quad (19)$$

$$\Gamma = \frac{e^{-rT} n(d_1)}{F\sigma\sqrt{T}} \quad (20)$$

$$\mathcal{V} = Fe^{-rT} n(d_1)\sqrt{T} \quad (21)$$

$$\Theta_{call} = -\frac{Fe^{-rT} n(d_1)\sigma}{2\sqrt{T}} + rFe^{-rT} N(d_1) - rKe^{-rT} N(d_2) \quad (22)$$

$$\Theta_{put} = -\frac{Fe^{-rT} n(d_1)\sigma}{2\sqrt{T}} - rFe^{-rT} N(-d_1) + rKe^{-rT} N(-d_2) \quad (23)$$

3.4 Rho: A Shortcut I Rely On

Because r only ever enters the Black-76 formula through the discount factor e^{-rT} multiplying the entire risk-neutral expectation, I get a very clean identity:

$$\rho = \frac{\partial V}{\partial r} = -T \cdot V \quad (24)$$

I implement `rho()` in `black76.py` as literally `-self.time_to_expiry * self.price()`, and I have a unit test in `tests/test_black76.py` confirming this against the model's own price.

4 Binomial Trees: CRR and Leisen-Reimer

I use these two tree constructions for the "American Call and Put" product bucket, implemented in `src/models/binomial_tree.py`. Both trees share the exact same backward induction and early-exercise logic; they only differ in how they compute the up-move factor u , down-move factor d , and risk-neutral probability p .

4.1 Cox-Ross-Rubinstein (CRR) Parameters

With n steps and $\Delta t = T/n$:

$$u = e^{\sigma\sqrt{\Delta t}} \quad (25)$$

$$d = \frac{1}{u} \quad (26)$$

$$p = \frac{e^{(r-q)\Delta t} - d}{u - d} \quad (27)$$

4.2 Leisen-Reimer (LR) Parameters

The LR construction is built so that the tree's implied risk-neutral probability matches the Black-Scholes d_1/d_2 terms directly, via the Peizer-Pratt inversion function:

$$h(z) = \frac{1}{2} + \text{sign}(z) \sqrt{\frac{1}{4} - \frac{1}{4} \exp \left[- \left(\frac{z}{n + \frac{1}{3} + \frac{0.1}{n+1}} \right)^2 \left(n + \frac{1}{6} \right) \right]} \quad (28)$$

I compute d_1 and d_2 once, using the full time to expiry T (not Δt), exactly as in the Black-Scholes-Merton section above. Then:

$$p = h(d_2) \quad (29)$$

$$p' = h(d_1) \quad (30)$$

$$u = e^{(r-q)\Delta t} \cdot \frac{p'}{p} \quad (31)$$

$$d = \frac{e^{(r-q)\Delta t} - pu}{1 - p} \quad (32)$$

I force the number of steps n to be odd in my `LeisenReimerTree` class, since the LR construction is designed around an odd step count so the middle terminal node lines up with the strike.

4.3 Backward Induction and Early Exercise

Regardless of which set of (u, d, p) I am using, I build the terminal payoffs at maturity, and then, for every step i from $n - 1$ down to 0 and every node j from 0 to i :

$$V_{i,j} = \max \left(\underbrace{e^{-r\Delta t} [pV_{i+1,j+1} + (1-p)V_{i+1,j}]}_{\text{continuation value}}, \underbrace{\phi(S_{i,j})}_{\text{intrinsic value, American only}} \right) \quad (33)$$

where $S_{i,j} = S_0 u^j d^{i-j}$ is the asset price at node (i, j) , and ϕ is the call or put payoff function. For a European contract, I skip the max with intrinsic value and just take the continuation value.

4.4 Greeks From the Tree

I read delta, gamma and theta directly off the tree's early nodes, which avoids having to bump-and-reprice the whole tree three separate times:

$$\Delta = \frac{V_{1,1} - V_{1,0}}{S_0 u - S_0 d} \quad (34)$$

$$\Gamma = \frac{\left(\frac{V_{2,2} - V_{2,1}}{S_0 u^2 - S_0 u d} \right) - \left(\frac{V_{2,1} - V_{2,0}}{S_0 u d - S_0 d^2} \right)}{\frac{1}{2}(S_0 u^2 - S_0 d^2)} \quad (35)$$

$$\Theta = \frac{V_{2,1} - V_{0,0}}{2\Delta t} \quad (36)$$

The Theta formula above works because node $(2, 1)$ has asset price $S_0 u d = S_0$ (one up move and one down move cancel out), so I am comparing the option value at the same spot price two time steps apart, which gives a clean central-difference-in-time estimate.

For vega and rho, I do not have a similarly clean closed form on the tree, so I bump volatility (or the risk-free rate) by a small amount, rebuild the whole tree, and take a central finite difference of the resulting prices. This is standard industry practice for tree-based vega/rho.

4.5 Early Exercise Boundary

I trace the early exercise boundary by scanning, at every time step before maturity, for the asset price closest to the strike where the tree's option value equals its intrinsic value (immediate exercise is optimal). I implement this in `early_exercise_boundary()`, and plot it in my "Early Exercise Boundary" chart.

5 Monte Carlo Engine

I use Monte Carlo simulation for three path-dependent product buckets: Asian options, Lookback options, and Basket options, implemented in `src/models/monte_carlo.py`.

5.1 Geometric Brownian Motion Discretization

I simulate the underlying under the risk-neutral measure using the exact lognormal discretization of GBM (not an Euler-Maruyama approximation), one time step at a time:

$$S(t + \Delta t) = S(t) \exp \left[\left(r - q - \frac{1}{2} \sigma^2 \right) \Delta t + \sigma \sqrt{\Delta t} Z \right], \quad Z \sim \mathcal{N}(0, 1) \quad (37)$$

I roll this recursion forward with an explicit Python for-loop over the number of time steps in a path, one multiplication at a time, rather than building the whole path with a vectorized cumulative product.

5.2 Antithetic Variates

For every set of standard normal draws $Z_1, \dots, Z_m$ I use to build one path, I also build a second path using $-Z_1, \dots, -Z_m$. Averaging the payoff of a path with its antithetic partner reduces the variance of my Monte Carlo estimator whenever the payoff function is a monotonic function of the path (true for Asian, Lookback and Basket calls/puts), because the two paths are negatively correlated by construction while sharing the same marginal distribution.

5.3 Discounted Price and Standard Error

Given M simulated payoffs $X_1, \dots, X_M$:

$$\hat{V} = e^{-rT} \cdot \bar{X}, \quad \bar{X} = \frac{1}{M} \sum_{i=1}^M X_i \quad (38)$$

$$\widehat{SE} = e^{-rT} \cdot \sqrt{\frac{s^2}{M}}, \quad s^2 = \frac{1}{M-1} \sum_{i=1}^M (X_i - \bar{X})^2 \quad (39)$$

and I report a 95% confidence interval as $\hat{V} \pm 1.96 \cdot \widehat{SE}$.

5.4 Payoffs I Implement

$$\text{Asian call : } \max \left(\frac{1}{m} \sum_{k=1}^m S(t_k) - K, 0 \right) \quad (40)$$

$$\text{Asian put : } \max \left(K - \frac{1}{m} \sum_{k=1}^m S(t_k), 0 \right) \quad (41)$$

$$\text{Fixed-strike lookback call : } \max \left(\max_k S(t_k) - K, 0 \right) \quad (42)$$

$$\text{Fixed-strike lookback put : } \max \left(K - \min_k S(t_k), 0 \right) \quad (43)$$

$$\text{Floating-strike lookback call : } S(T) - \min_k S(t_k) \quad (44)$$

$$\text{Floating-strike lookback put : } \max_k S(t_k) - S(T) \quad (45)$$

5.5 Basket Options and Correlated Paths

For a basket of N underlyings with correlation matrix ρ , I factor $\rho = LL^\top$ using the Cholesky decomposition (computed by hand with explicit nested loops in `_cholesky_decomposition()`), then at every time step I draw N independent standard normals ϵ and correlate them as $Z = L\epsilon$, before applying the usual GBM update to each asset i separately using Z_i . The basket payoff is:

$$\text{Basket call} = \max \left(\sum_{i=1}^N w_i S_i(T) - K, 0 \right) \quad (46)$$

5.6 Greeks by Finite Differences (Bump-and-Reprice)

Because Monte Carlo does not give me an analytic Greek, I estimate every Greek by bumping one input, rebuilding the model, and repricing, always reusing the same `random_seed` between the bumped and unbumped simulations (the "common random numbers" technique). Sharing the random seed cancels out most of the simulation noise between the two prices, which is what makes a finite-difference Greek usable at all on top of a noisy Monte Carlo price:

$$\Delta \approx \frac{V(S+h) - V(S-h)}{2h} \quad (47)$$

$$\Gamma \approx \frac{V(S+h) - 2V(S) + V(S-h)}{h^2} \quad (48)$$

$$\mathcal{V} \approx \frac{V(\sigma+h) - V(\sigma-h)}{2h} \quad (49)$$

$$\Theta \approx \frac{V(T-h) - V(T)}{h} \quad (50)$$

$$\rho \approx \frac{V(r+h) - V(r-h)}{2h} \quad (51)$$

6 Finite Difference PDE for Barrier Options

I use an explicit finite difference scheme for the "Barrier Options" product bucket, implemented in `src/models/finite-difference.py`.

6.1 The Black-Scholes PDE

Every one of my pricing models ultimately prices a claim on the same underlying dynamics, which satisfy the Black-Scholes partial differential equation:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + (r-q)S \frac{\partial V}{\partial S} - rV = 0 \quad (52)$$

6.2 Explicit Finite Difference Discretization

I discretize S into M steps of size $\Delta S = S_{max}/M$ and t into N steps of size $\Delta t = T/N$, and write V_i^j for the option value at price node $S_i = i\Delta S$ and time node j (where $j = N$ is maturity and $j = 0$ is today). Using a forward difference in time and central differences in S , I get the explicit update rule:

$$V_i^j = a_i V_{i-1}^{j+1} + b_i V_i^{j+1} + c_i V_{i+1}^{j+1} \quad (53)$$

with coefficients

$$a_i = \frac{1}{2}\Delta t (\sigma^2 i^2 - (r - q)i) \quad (54)$$

$$b_i = 1 - \Delta t (\sigma^2 i^2 + r) \quad (55)$$

$$c_i = \frac{1}{2}\Delta t (\sigma^2 i^2 + (r - q)i) \quad (56)$$

I chose the explicit scheme (rather than implicit or Crank-Nicolson) specifically because this update rule is a single readable line per node, at the cost of needing a fine enough time grid to stay numerically stable.

6.3 Stability Condition

The explicit scheme is only stable if Δt is small enough relative to ΔS . I use the conservative sufficient condition:

$$\Delta t \leq \frac{1}{\sigma^2 M^2 + r} \quad (57)$$

and in `_stable_num_time_steps()` I automatically increase the number of time steps beyond whatever the caller requested if this bound would otherwise be violated, so the scheme cannot silently blow up.

6.4 Boundary and Barrier Conditions

At the edges of my price grid, I use:

$$\text{Call: } V(0, t) = 0, \quad V(S_{max}, t) = S_{max}e^{-q(T-t)} - Ke^{-r(T-t)} \quad (58)$$

$$\text{Put: } V(0, t) = Ke^{-r(T-t)}, \quad V(S_{max}, t) = 0 \quad (59)$$

For a knock-out barrier, I force $V_i^j = 0$ at every node whose price has crossed the barrier, at every time step (I check this immediately after computing the continuation value, before moving to the next node). This models a continuously monitored barrier.

6.5 Knock-In via In/Out Parity

I price knock-in barriers using the standard market identity:

$$V_{\text{knock-in}} = V_{\text{vanilla}} - V_{\text{knock-out}} \quad (60)$$

This holds because holding both the knock-in and the matching knock-out version of a barrier option is, payoff for payoff, identical to holding the vanilla option: on any given path, exactly one of the two barrier contracts pays off, depending on whether the barrier was breached.

6.6 Greeks From the Grid

Similarly to my binomial trees, I read delta, gamma and theta directly off neighboring grid nodes at today's time slice ($j = 0$) around the interpolated spot index:

$$\Delta \approx \frac{V_{i+1}^0 - V_{i-1}^0}{2\Delta S} \quad (61)$$

$$\Gamma \approx \frac{V_{i+1}^0 - 2V_i^0 + V_{i-1}^0}{\Delta S^2} \quad (62)$$

$$\Theta \approx \frac{V_i^1 - V_i^0}{\Delta t} \quad (63)$$

and I fall back to bump-and-reprice whenever the spot price sits too close to the edge of the grid to safely read a neighboring node, and always for vega and rho.

7 Implied Volatility Calibration

I implemented my calibration engine in `src/calibration/implied_volatility.py` to back out the volatility that reprices a given market option price under Black-Scholes-Merton.

7.1 Newton-Raphson

Starting from an initial guess σ_0 , I iterate:

$$\sigma_{k+1} = \sigma_k - \frac{C(\sigma_k) - C_{\text{market}}}{\mathcal{V}(\sigma_k)} \quad (64)$$

until $|C(\sigma_k) - C_{\text{market}}|$ is below my tolerance, using the model's own vega as the derivative.

7.2 Bisection Fallback

If Newton-Raphson steps outside my $[\sigma_{\min}, \sigma_{\max}]$ bounds, or vega becomes too small to divide by safely, I fall back to a bisection search on the same interval, which is guaranteed to converge (much more slowly, but reliably) as long as the market price is bracketed between the model prices at $\sigma_{\min}$ and $\sigma_{\max}$.

7.3 Rejecting Impossible Prices

Before running either root-finding method, I check the market price against the no-arbitrage bounds from Section 2.5. If the market price is outside those bounds, no volatility (however extreme) could ever reproduce it under Black-Scholes-Merton, so I raise an error immediately instead of letting a root finder search forever.

8 Scenario Analysis

My scenario engine, in `src/scenario_analysis/scenario_engine.py`, clones a pricing model with `copy.deepcopy`, mutates the cloned copy's stored attributes directly, and reprices. Because every model in my platform recomputes its price from its own stored attributes, this has the exact same effect as fully re-running the model with a shocked input, without needing model-specific reconstruction logic. I apply the following shocks, matching Phase 9 of my project specification:

- Underlying price: $\pm 1\%$, $\pm 5\%$, $\pm 10\%$
- Volatility: ± 5 , ± 10 vol points
- Interest rate: ± 50 , ± 100 basis points
- Time decay: 1, 7, 30 calendar days forward
- Basket correlation (basket products only): $\pm 5\%$, $\pm 10\%$ pairwise correlation
- Barrier level (barrier products only): $\pm 5\%$, $\pm 10\%$ barrier level

9 Model Validation

My validation suite, in `tests/test_model_validation.py`, checks every one of the following properties, matching Phase 12 of my project specification:

- **Put-call parity** holds exactly (to floating point precision) for Black-Scholes-Merton and Black-76.
- **No-arbitrage bounds** are respected by every European closed-form price.
- **American price $\geq$ European price**, checked on both CRR trees with early exercise enabled versus disabled.
- **CRR and LR convergence**: the pricing error versus the Black-Scholes benchmark shrinks as I increase the number of tree steps.
- **Monte Carlo convergence**: the standard error of my price estimate shrinks as I increase the number of simulated paths.
- **Stable Greeks**: a small perturbation to an input produces a correspondingly small change in the Greeks, not an erratic jump.
- **Correct barrier behavior**: a knock-out option is worth less than (or numerically close to) the vanilla option, and a barrier set essentially at the money knocks out almost all of the option's value.

10 Limitations I Am Aware Of

- My finite difference PDE solver uses a first-order accurate explicit scheme. A production barrier pricer would typically use Crank-Nicolson for second-order accuracy in time, at the cost of needing a tridiagonal solver.
- My Monte Carlo engine uses plain antithetic variates as its only variance reduction technique. Control variates (pricing the equivalent European option analytically and using it to reduce variance on the path-dependent payoff) would tighten my confidence intervals further.
- My barrier model only supports a single, continuously monitored barrier. Discrete monitoring (barrier only checked at specific dates) and double-barrier structures are not implemented.
- My market data module falls back to manually entered data whenever `yfinance` or network access is unavailable, which is expected in a sandboxed environment but means live implied volatility surfaces are not something I can guarantee end to end without a live connection.

11 Future Enhancements I Would Consider

- Adding a Crank-Nicolson solver alongside my explicit scheme for the barrier pricer.
- Adding a local volatility or stochastic volatility model (e.g. Heston) for a more realistic volatility smile.
- Adding control variates to my Monte Carlo engine.
- Extending my basket option engine to support discrete monitoring dates for Asian and Lookback products with fewer than daily observations.